Session 3: Verification Through to the Paper

Estimation, Robustness, LaTeX, Git, and Results to Paper

Brady Allardice

UPF & IBEI

Where We Are

Session 1: What Claude Code is and how it works.

Session 2: Staying in control: context, plan mode, and verification. You learned the Swiss franc data, wrote your own expectations for the merge, had Claude Code write tests from them, and wrote your own `CLAUDE.md` rules.

Session 3: From a verified dataset to a paper.

Today you will:

1. Finish the Swiss franc project: estimate, account for every observation, run robustness checks, produce a LaTeX table
2. Set up LaTeX in VS Code and learn git as the undo button for agent edits
3. Put results into a paper, have Claude Code write the text, and check every number against the table

A number in a paper is a claim. Today you check it before it leaves the project.

Start-of-Session Ritual

The same one command as every session. Copy it from the top of **github.com/bradyallardice/claude-code-workshop**.

Mac (Terminal)

```
curl -fsSL https://raw.githubusercontent.com/
  bradyallardice/claude-code-workshop/student/
  tools/start_session.sh | bash -s -- 3
```

Windows (PowerShell)

```
irm https://raw.githubusercontent.com/
  bradyallardice/claude-code-workshop/student/
  tools/start_session.ps1 | iex
```

- **Work only in** `/my_workspace`**.** The course repo is a mirror that the command resets.
- Files you have **not** edited are updated; files you **have** edited are never touched, and the newer version appears beside them as `<name>.new`. This includes your `session_2` folder.
- **Then reopen your** `session_2` **copy** in a new VS Code window (File → New Window → Open Folder). Part 2 continues in it.

▸ Command fails?

Check-In: Claude Code in Your Work

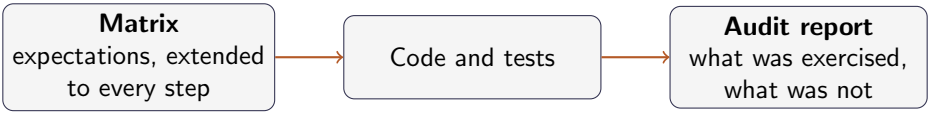
Since Session 2, have you tried Claude Code on your own research?

- **What for:** what kind of tasks?
- **Pain points:** what was frustrating or hard?
- **Trust:** are you confident in the code it produces?
- **Decisions:** did it make a choice you did not ask for? Did you notice at the time?

From Expectations to a Matrix

In Session 2 you wrote `expectations.md` for the merge, and Claude Code wrote tests from it. That was one component.

Today the same discipline covers the whole analysis, with a record of it:



The matrix makes each expectation *accountable*: where it came from, what independent source checks it, what exact result passes. The audit report is the matrix after the work, with the gaps showing.

Building the Matrix: Where Requirements Come From

Write it **before** the code, from sources independent of the code, as you did for the merge.

Source	What it gives you
Research question and design	Reference category, sample, weights, what the estimand is
Codebook and raw data	Keys, ranges, missing-value codes, valid-but-odd values (income = 0)
Each pipeline step	What must be true after the merge, after each recode, after each model
The claim you will make	What must hold for the number in the paper to mean what the text says

For each requirement, ask three things:

1. What would a violation look like?
2. What source other than my code tells me the right answer?
3. What exact result counts as a pass?

A Weak Row and a Strong Row

	Weak	Strong
Requirement	"The exposure measure is right"	Each zone's robots per 1,000 workers equals its robot count divided by its working-age population, in thousands
Test	Re-runs the build code and compares with its own output	Recomputes the ratio from the two raw files, without the pipeline code
Passing condition	"Looks reasonable"	Matches a hand calculation for five named zones within 1e-6; no zone has a negative value

Invented example: a made-up commuting-zone project, not yours.

Three ways a row goes weak

The requirement is too vague to fail. The test reuses the code it checks, so it passes by construction (Session 2's independence rule). The condition is a judgment, not a number.

Hold your own expectations up against these three.

The Matrix Format

One row per requirement. Five columns:

Column	What goes in it
Requirement	One sentence, in research terms, that can be true or false
Source	Where you got it: codebook, design, research question, raw data
Test	The check, and what it computes from
Independent truth	The value or fact the check compares against, obtained without the pipeline
Passing condition	An exact number, set, or tolerance

A row with an empty *Independent truth* cell is a check that cannot catch the code's own mistakes.
Next slide: a filled-in example.

The Requirements-to-Tests Matrix

Requirement	Test	Passing condition
Panel is balanced: every zone appears in both election years	Key-count test on the raw returns	722 zones $\times$ 2 years = 1,444 rows; no (zone, year) pair repeated
Exposure built as defined	Independent recomputation	Robots per 1,000 workers matches a hand calculation for five named zones, within 1e-6
Standard errors are clustered by state	Specification test	Output reports 49 clusters, the number of distinct states in the raw file
Zones with missing manufacturing employment are dropped, and counted	Sample accounting	1,444 $-$ 38 dropped (missing employment) = 1,406 used
Change in vote share uses the right years	Independent recomputation	Three zones' 2016-to-2020 change, taken from the official returns, equals the pipeline's within 0.001

Invented example: a made-up study of robot adoption and populist voting across commuting zones. Your rows will have different content.

Write the matrix before the code. A requirement with no test is unverified.

Writing the Audit Report: What to Ask of Each Section

Section	Ask
Tests run	Were any skipped? Does the count match what you expect?
Requirements	Does every matrix row have a test? Is any row missing?
Not tested	Is the list honest? What did Claude not check, and why?
Numerical checks	Was the independent value computed without the pipeline? Is the tolerance stated?
Needs your call	Which choices could a referee challenge? Did Claude make any silently?
Not covered	What can passing tests never show? (identification, construct validity)

Read it as an adversary: the report is only useful if its gaps are as visible as its passes.

The Audit Report: Evidence, Not "All Tests Passed"

```
## Audit report
Tests run: commands; passed / failed / skipped
Requirements: each requirement -> its test
Not tested: anything without a test, and why
Numerical checks: our value vs. independent value
Needs your call: choices Claude proposed
Not covered: identification, construct validity
```

What the report is for

Not for Claude Code to declare the research correct. It documents what was exercised, against which expectation, and what remains uncertain.

An Audit Report, Filled In (Swiss Franc Project)

```
## Tests run          126 passed | 0 failed | 0 skipped | 5 known issues
## Requirements       R1 Merge keeps every respondent ..... 7 tests, pass
                      R3 Control is the reference arm ..... 2 tests, pass
                      R5 N explained for every model ..... 15 tests, pass
## Not tested         Stata and R versions; interaction coefficients (N only)
## Numerical checks                pipeline    independent   tolerance
  intercept vs logit(share)        -0.218726   -0.218726     1e-5
  info coef, full controls          0.266587    0.266587     2e-6
## Needs your call
  - DK coded 0, not missing (230 respondents): the codebook's definition,
    but a substantive choice.
  - Income + left-right drops ~43% of respondents: the tests reconcile
    the drop; they do not say complete cases are acceptable.
## Not covered       identification; construct validity of the outcome
```

Notice what the report does: it names two judgment calls and what it cannot show, instead of declaring the work correct.

Two More Safeguards

Replicate before you extend

1. Give Claude Code the original code or description
2. Ask it to replicate a key table
3. Compare with the published result
4. Only then begin your extension

If you cannot reproduce the baseline, nothing built on it is interpretable.

Use a second agent as a critic

1. Agent 1 writes the code
2. Open a new conversation (Agent 2) and ask: "Read the analysis script and identify any bugs, silent data loss, or decisions that could change the results."

A fresh session is an independent reviewer (Session 2). This one reads the code; a stricter version would work from the codebook alone and never see it.

Treat your agents like co-authors who check each other's work, not a single assistant you trust unconditionally.

What to Delegate, What to Verify, What to Keep

Delegate freely

Boilerplate & docs
File management†
Reformatting code

Delegate + verify

Data loading & cleaning
Data merges
Variable construction
Summary statistics
Robustness variants
Visualization
LaTeX tables

Do not delegate

Choosing specifications
Interpreting results
Identification strategy
Sample selection
Causal claims

† Be careful with file management — Claude Code can delete files. Always review before approving.

The test: *If a referee questioned this choice, who would need to answer — you or the RA?*

Today's exercise focuses on the middle column: tasks where Claude Code is productive but where you must verify every output.

Before You Start: Reopen Your Session 2 Folder

Part 2 continues the project you started in Session 2. Open `my_workspace/session_2` in a **new VS Code window**.

Check that these exist:

- `CLAUDE.md` with your own verification rules, and your `expectations.md`
- Your merge script and `output/merged_survey.csv`
- Your merge tests, and they pass

If your merge did not finish: ask Claude Code to finish it and run its tests first. Do not estimate on a merge you have not checked. Give this at most five minutes.

Start a new conversation. The old one carries Session 2's context.

Exercise Part 2 (1/2): Audit and Extend Your Expectations (9 min)

In your `session_2` folder, **before any model is estimated**.

1. **Audit your own** `expectations.md` (3 min). For each expectation, ask: where did it come from? what independent source checks it? what exact result passes?
   - Mark each one that is too vague to fail, reuses the code, or rests on a judgment. Rewrite those.
2. **Turn it into a matrix** (3 min): `docs/requirements_matrix.md`, five columns: requirement, source, test, independent truth, passing condition. Add **at least four rows for the model**. The requirements are yours.
   - Ask Claude Code to *review* your rows for the three weak patterns. It reviews; it does not write them.
3. **Tests for the new rows** (3 min), as in Session 2: tests first, read them before any model code. They should fail now. Does each expected value come from the raw files?

Exercise Part 2 (2/2): Specify and Estimate (16 min)

3. **Base model:** Logistic regression of `supports_intervention` on `treatment`, with survey weights. Tell Claude Code the reference category. Run your matrix tests.
4. **Add FX exposure.** Did the sample size change? Why?
5. **Add demographic controls:** age, female, education, urban/rural, income, left-right.
   - Predict your N first. Add a matrix row and a test for it, then run the model.
6. *If time:* interact treatment with FX exposure.

For every model, reconcile N: rows in − rows dropped (by reason) = N used. **If a test fails, fix the code or the row, and say which.**

Part 2 Debrief: What Happened to Your Sample?

- How many observations did you have in each model? Did your prediction match?
- Which variable cost you the most observations? Where did the rows go?
- Did Claude Code mention the change in sample when it added the controls, or did you find it?
- Could you reconcile rows in − rows dropped = N used for every model?

The point

Adding control variables can silently change your sample.

The Robustness Workflow

You specify the core analysis.
Claude Code generates the variations.
You verify each one.

Claude Code never chooses what robustness checks to run.

For each specification, check:

- Does it use the right sample? (What is N?)
- Does the code match what you asked for?
- Are the coefficients in a plausible range?
- Do the signs on control variables make sense?

LaTeX Tables: Bringing Results Into Your Paper

Until now, your results are printed to the console or saved as CSVs.
In your actual workflow, they need to end up in a paper.

Why this matters:

- Claude Code can write LaTeX tables directly from regression output
- The table lives in your project directory, under version control
- When you update the analysis, Claude Code updates the table
- No manual reformatting, no copy-paste errors

First: how to write, compile, and debug LaTeX inside VS Code, so you can build and read that table. Then the paper it goes into.

The bigger idea

The more of your research that lives inside the project directory, the more Claude Code can operate on. Code, data, output, tables — and eventually the paper itself.

The Problem with Overleaf

Most researchers keep their paper in a separate tool.

- Code lives in a project directory (or on your Desktop)
- Data lives next to the code (maybe)
- The paper lives in Overleaf (or Word, or Google Docs)

What this means for Claude Code:

- It cannot read your paper
- It cannot check the methods section against your code
- It cannot update a table when the analysis changes
- It cannot commit changes to your paper alongside code changes

The Limitation

If Claude Code cannot see it, Claude Code cannot operate on it.
Your paper is the biggest piece of your project that is currently invisible.

The Solution: LaTeX in VS Code

Bring your paper into the project directory.

- Your `.tex` file lives alongside `scripts/` and `data/`
- Claude Code can read it, write it, and edit it — just like any other file
- Changes to the paper are tracked by git, just like code changes
- Coauthors review changes through version control, not "track changes"

What becomes possible:

- Claude Code writes a methods section and you verify it against the code
- The analysis changes $\rightarrow$ the table updates $\rightarrow$ the paper updates
- Every version of the paper is recoverable through git

The Point

This is not about LaTeX vs. Word. It is about bringing your paper into the agentic workflow.

LaTeX in VS Code: The Setup

What you need (should be installed from pre-work):

1. A TeX distribution: TeX Live (Linux/Windows) or MacTeX (Mac)
2. The **LaTeX Workshop** extension in VS Code

What you get:

- Syntax highlighting for `.tex` files
- One-click compilation to PDF
- PDF preview in a split pane — side by side with your code
- Error messages in the VS Code terminal

The workflow:

Edit `.tex` → Save → Auto-compile → PDF updates in the preview pane.

You never leave VS Code. Your paper is right next to your code.

New to LaTeX?

LaTeX Workshop: Essential Commands

Three ways to compile:

Keyboard Shortcuts (Mac):

```
Cmd+Alt+B Compile to PDF
Cmd+Alt+V Open PDF preview
```

Keyboard Shortcuts (Windows/Linux):

```
Ctrl+Alt+B Compile to PDF
Ctrl+Alt+V Open PDF preview
```

Or click the green button:



Once the preview is open:

- Right-click the PDF preview tab and select `Move into Side Panel`
- Now you have code on the left, PDF on the right

Optional: Auto-compile on save

Add to VS Code settings: `"latex-workshop.latex.autoBuild.run": "onFileChange"`

Live Demo: Building a Paper from Scratch

What I'm going to show you:

I have a folder with pre-generated outputs from a fake analysis of *this course*:

- Summary statistics table (`.tex`)
- Regression results table (`.tex`)
- Four figures: enrollment, satisfaction, attrition, earnings effect

Starting from nothing, I'll use Claude Code to:

1. Create a blank `course_paper.tex` file
2. Write methods and data sections describing the study
3. Pull in the pre-generated tables with `\input{...}`
4. Embed the figures with `\includegraphics{...}`
5. Compile to PDF and fix any errors that come up

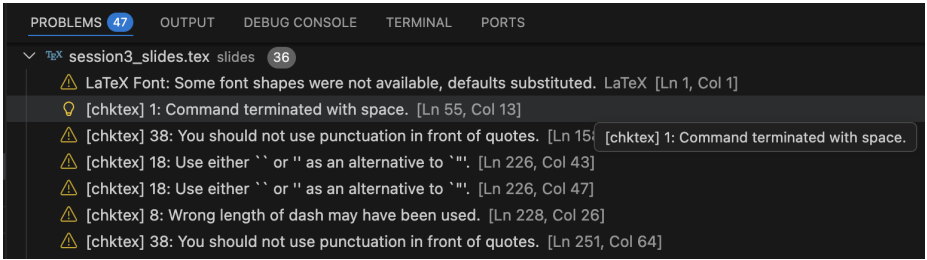
Watch how Claude Code reads the Problems pane and iterates without me copy-pasting errors.

LaTeX Errors and the Problems Pane

When you compile, LaTeX Workshop checks for errors and syntax issues. They appear in the **Problems** pane at the bottom of VS Code.

To view the Problems pane:

- Keyboard: `Ctrl+Shift+M` (or `Cmd+Shift+M` on Mac)
- Menu: `View` → `Problems`



Errors and warnings appear here with line numbers. Click any error to jump to that line in your `.tex` file.

Why All This Matters for Claude Code

The payoff: Everything is in one place. Your code, your data, your LaTeX document, and your compilation errors are all visible to Claude Code.

Claude Code can:

- **See the error:** Claude Code reads the Problems pane and helps you fix it
- **Write the LaTeX:** Describe what you want (a table, a figure reference, a citation) and Claude Code writes it
- **Check consistency:** Ask Claude Code to verify that your methods section matches your actual code
- **Iterate quickly:** Write → Compile → Show error to Claude Code → Revise → Repeat

The Workflow

Your `.tex` file is no longer separate from your research. It is part of your project, subject to the same verification and version control as your code.

Exercise Part 3: Robustness, Output, and the Audit Report (25 min)

7. **Robustness variants:** with and without weights; different control sets (track N); "any information" vs. control; a subsample. Add a matrix row for each: what must change, and what must stay the same?

8. **LaTeX table,** all specifications side by side, compiled to PDF. Output-manifest test: every specification appears with its N, and the cells match the model output.

9. **Audit report,** `output/audit_report.md`, built from your matrix: each row → its test → its result. A row with no test goes under *Not tested*. Read it as an adversary: are its gaps as visible as its passes?

10. **Revise your** `CLAUDE.md`**.** Pick one gap the report exposed and write the rule that would have caught it, in your own words. Then compare your rules with a reference protocol. ▸ Reference protocol

11. **Commit** scripts, tests, matrix, output, table, report, and `CLAUDE.md`. (Git is next; if it is not set up yet, do it after the git frames.)

What to Watch For During the Exercise

This exercise is designed to surface moments where Claude Code makes choices for you.

- **Reference categories:** Did it set treatment to "control" or pick alphabetically?
- **Missing data:** When you added controls, how many observations did you lose? Did Claude Code mention this?
- **Survey weights:** Did it use them unless you told it to?
- **Income = 0:** "No income" is coded as 0. How did Claude Code treat it?
- **Tests:** Did it copy the pipeline's own output in as an expected value, or edit one to get a pass?

These are the moments where your judgment matters.

The workflow is: notice the decision, direct Claude Code explicitly, then verify the output.

Debrief

Discussion questions:

1. How many observations did you lose when you added controls? Did Claude Code tell you?
2. How many of your robustness specifications were correct on the first try?
3. What did you catch in the plan that you would have missed in the output?
4. Did any test fail? Did Claude Code fix the code, or change the test?
5. Which matrix row did you have to rewrite, and why? Did any row end up with no independent truth?

Key takeaway

The tests, the assertions, and the plan review are not extra work — they are the workflow. The 30 seconds it takes to read a plan or check a sample size can save hours of debugging downstream.

Why Git Matters for Agentic Coding

The problem: Claude Code modifies your files directly.

- It might change a file you did not ask it to change
- It might overwrite working code with something broken
- It might make 10 changes when you only wanted 3

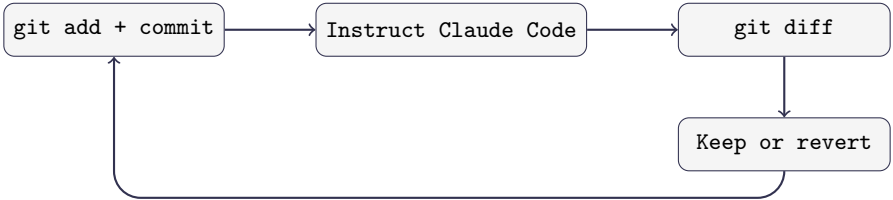
Without git: You try to manually undo changes. You are not sure what the file looked like before. You lose work.

With git: You see exactly what changed, line by line. You revert anything you do not want. You never lose work.

The Rule

Never let Claude Code touch your project without a recent commit to fall back on.

The Minimum Viable Git Workflow



Command	What it does
`git add -A && git commit -m "msg"`	Snapshot your current state
`git diff`	See what changed since last commit
`git checkout -- file.py`	Undo changes to a specific file
`git log --oneline`	See your commit history

Command 1: `git init`

What it does: Turns a regular folder into a git project.

```
git init
```

- You run this **once**, when you start a new project
- It creates a hidden `.git/` folder that stores your project's history
- Nothing else changes — your files stay exactly as they are

Think of it as saying: "I want to start tracking this folder."

Note

Your session folders are copies without a `.git` folder, so each project needs its own `git init`. `git status` tells you whether a folder is already tracked.

Command 2: `git add`

What it does: Selects which files to include in your next save point.

```
git add -A
```

- `-A` means "add everything that changed" — new files, modified files, deleted files
- This does not save anything yet. It just marks what will be included in the next commit.

You can also add specific files instead of everything:

```
git add scripts/analysis.py
```

Think of it as putting files in a box before sealing it. `git add` fills the box; `git commit` seals it.

Command 3: `git commit`

What it does: Saves a snapshot of everything you added.

```
git commit -m "Clean merge script, verified row counts"
```

- `-m` stands for "message" — a short note describing what this snapshot contains
- The message goes in quotes. Write something your future self will understand.
- After committing, you have a permanent save point you can always return to.

In practice, you can always run add and commit together (but you don't have to):

```
git add -A && git commit -m "your message"
```

The `&&` just means "run the second command after the first one finishes."

Command 4: `git diff`

What it does: Shows you exactly what changed since your last commit.

```
git diff
```

The output looks like the diff view you already know from VS Code:

- Lines starting with + were added (green in VS Code)
- Lines starting with − were removed (red in VS Code)
- Everything else is unchanged context

When to use it: After Claude Code makes changes, run `git diff` to see *everything* it touched — not just the file it told you about.

Tip

You can also ask Claude Code: "Run git diff and explain each change you made."

Command 5: `git log`

What it does: Shows you the history of all your commits.

```
git log --oneline
```

Output looks like:

```
a3f7b21 Add robustness checks
e5c9d04 Clean merge script, verified row counts
b1a2c33 Initial commit
```

- Each line is one commit — one save point
- The letters and numbers on the left are the commit ID
- The message on the right is what you wrote with `-m`
- Most recent commits are at the top

This is your project's timeline. You can always see what you did and when.

Command 6: `git checkout`

What it does: Undoes changes by restoring files to a previous committed state.

Undo uncommitted changes to one file:

```
git checkout -- scripts/analysis.py
```

Undo all uncommitted changes:

```
git checkout -- .
```

Go back several commits (rewind the whole project):

```
git log --oneline # find the commit you want
git checkout <commit-hash> # jump to that commit
git checkout main # come back to the latest
```

Or use `HEAD~N` to go back `N` commits from your current position:

```
git checkout HEAD~3 # go back 3 commits
```

`git checkout`: Your Undo Button

The mental model:

- `git checkout -- .` → undo *uncommitted* work
- `git checkout <hash>` → travel back to any previous commit
- `git checkout main` → return to the latest state

Recovering if something goes wrong:

```
git reflog # shows every action you've taken
```

Even changes you thought were gone are usually recoverable.

This all only works if you committed first

Checkout lets you jump between snapshots — but you need snapshots to jump to. That is why you always commit before instructing Claude Code.

Putting It Together: The Cycle

Before every interaction with Claude Code:

1. `git add -A && git commit -m "message"` — save your current state
2. Give Claude Code an instruction
3. `git diff` — see what it changed
4. Decide:
   - Happy? → `git add -A && git commit -m "message"`
   - Not happy? → `git checkout -- .`
5. Repeat

The Mindset Shift

Commits are not milestones. They are save points — like pressing Ctrl+S. Commit early, commit often, commit before every interaction with Claude Code.

.gitignore: Telling Git What to Skip

A `.gitignore` file lists patterns for files git should never track. Place it in the folder where you ran `git init`.

Paths are relative to that folder. Example:

```
my_project/ <- run "git init" here
  .gitignore <- goes here
  code/
    analysis.py
  data/ <- put in .gitignore
    raw/
    processed/
  output/ <- put in .gitignore
```

Always include in .gitignore:

```
.DS_Store
.env <- usually contains sensitive information
*.pyc
__pycache__/
*.aux
*.log
```

Then add: `data/raw/`, `*.csv.gz`, any large files.

Git and Data: What Not to Track

Git is designed for **code and text files** — things that change line by line.
It is *not* designed for large data files.

Commit:

- Scripts (`.py`, `.R`, `.do`)
- LaTeX files (`.tex`, `.bib`)
- Documentation (`CLAUDE.md`)
- Small data files (under a few MB)

Do not commit:

- Large datasets (50MB+)
- Output you can regenerate
- Sensitive data (PII, API keys)

Rule of Thumb

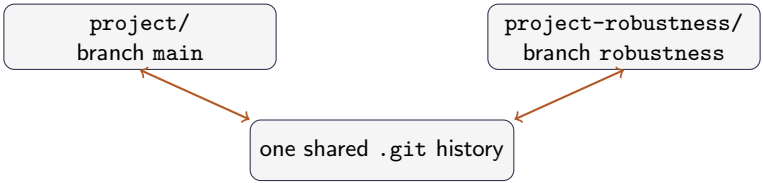
Commit what a collaborator needs to *understand* your project. Skip what they could *regenerate* by running the code.

Worktrees: Two Folders, One History

The problem: two Claude Code sessions in one folder edit the same files. Last writer wins (Session 2), and one `git diff` mixes both sessions' changes.

A worktree is a second folder on its own branch, sharing the same git history. A branch is a named line of commits.

```
git worktree add ../project-robustness -b robustness
git worktree list
```



Parallel Agents With Worktrees

The workflow

1. Commit your current work
2. One worktree per task
3. Open each folder in its own VS Code window, with Claude Code in each
4. Review and commit in each
5. Merge back from the main folder
6. Clean up

```
git merge robustness
git worktree remove ../project-robustness
```

Watch for

- **Ignored files are not copied.** `data/`, `.env`, and outputs in `.gitignore` are missing in the new folder
- **One branch, one worktree**
- **Same lines edited in both** means a merge conflict ▸ Conflicts
- **Two sessions cost twice** the tokens

Here: one worktree estimates the robustness variants while another drafts the paper section. Different files, so they merge cleanly.

If Your Coauthors Are on Overleaf

Moving your paper into VS Code does not mean abandoning your coauthors.

Overleaf supports git sync. Overleaf itself acts as the git remote, so you push and pull like any other repository. No GitHub account is involved.

The setup:

1. Enable git sync on your Overleaf project (requires a paid plan)
2. Clone the Overleaf remote to your local machine
3. Open the `.tex` files in VS Code and work with Claude Code as normal
4. Push your changes back to Overleaf when ready; coauthors see them immediately

Result

You get Claude Code and version control. Your coauthors get Overleaf.
Nobody has to change their entire workflow at once.

The Hybrid Workflow

You (VS Code + Claude Code):

- Edit `.tex` locally, ask Claude Code to update tables, fix references, rewrite paragraphs
- Commit changes to git — every edit is tracked and reversible
- Push to Overleaf when a section is ready for coauthor review

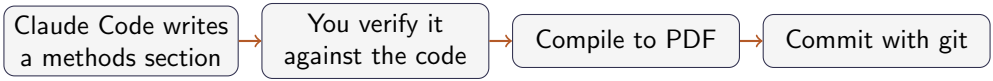
Your coauthors (Overleaf):

- Review and comment in the Overleaf interface as usual
- Make edits directly in Overleaf
- Their changes are commits you can pull locally

Caveat: merge conflicts are possible if you and a coauthor edit the same lines simultaneously — the same risk as any shared git repo. Pull before you push.

◂ Back matter

Demo: The Full Loop



Everything happens in VS Code.

Everything is version-controlled.

Everything is visible to Claude Code.

Exercise: Results → Paper (25 min)

Open `my_workspace/session_3/results_to_paper` in a new VS Code window. It holds `paper.tex`, `results/regression_results.csv`, and `scripts/render_regression_table.py`. Everyone starts from the same results.

1. **Git first.** `git init`, have Claude Code write a `.gitignore` for LaTeX build files, then commit.
2. **Render the table.** Have Claude Code run the render script and compile `paper.tex`. Open the CSV and the `.tex` fragment yourself and check that they agree. Commit.
3. **Results paragraph.** Commit first. Have Claude Code write it from the table. VS Code shows the diff before applying it: read it, then accept or reject.
4. **Cross-check.** Next slide.

Commit → instruct → diff → decide. Reject at least one change.

Exercise: Cross-Check Every Number

In a **fresh conversation**, so the checker has not just written the paragraph:

```
Read paper.tex and tables/regression_table.tex.
List every number in the Results paragraph, and the
table cell each one comes from. Then list every claim
(significant, robust, larger, smaller) and say whether
the table supports it. Do not edit any file.
```

Then answer for yourself:

- Did the check find anything you missed when you read the diff?
- What can this check *not* catch? (Is the table itself right? Is "causal" justified?)

Finish with a commit. `git log --oneline` should show your history.

Results → Paper: Debrief

1. Which sentences in the paragraph went beyond what the table supports?
2. What did you reject, and what did the commit make cheap?
3. Did the cross-check find something the diff did not, or the reverse?

Key takeaway

Claude Code can write the paragraph and check the numbers. It cannot tell you whether the table is the right one to report, or whether the sentence over-reads it. That stays with you.

When It Goes Wrong: Four Patterns

These are normal, not rare. Recognize the pattern early, because the right response differs.

Pattern	What you see	First response
Looping	The same fix again and again; the error does not change	Stop. Give it new information
Confidently wrong	Reports success; the numbers are wrong	Check against something it did not produce
Scope creep	Edits things you did not ask about	Read the diff; reject the extras
Coherence loss	Late on, it contradicts earlier decisions	Decisions into a file; new conversation

Pattern 1: Looping

What it looks like

- The same error after every "fix"
- Two changes alternating: A, then B, then A

Why

Each failed attempt anchors the next. No new information has come in.

What to do

- Stop after the second repeat
- Ask what it tried and why each failed
- Give it something new: the log, the full error

Example: a table will not compile, and a package is added, removed, and added again.

```
Stop. List what you tried and why each failed.
Then read paper.log and tell me the first error.
```

Pattern 2: Confidently Wrong

What it looks like

- "All tests pass" but the numbers are off
- Tests that agree with the code, because it wrote both

What to do

- Ask for evidence, not assurance
- Check against something it did not produce
- Read the tests

Why

It reports what it meant to do, not what the code did.

Example: a clustered model reported default standard errors. Comparing the cluster count in the output with the number of states in the raw file caught it.

```
Count the distinct states in the raw file and
compare with the cluster count in the model output.
```

Pattern 3: Scope Creep

What it looks like

- The diff touches files you did not mention
- A reference category or sample changed "while it was there"

What to do

- Commit first
- Read the diff; reject the extras
- Narrow the prompt: change X, touch nothing else

Why

A broad request is read broadly.

Example: "add income" also changes the reference category.

```
Change only the model formula in estimate.py.
Do not edit any other file or line.
```

Pattern 4: Coherence Loss

What it looks like

- A decision from early on is forgotten (the join type)
- A bug you already fixed comes back

Why

The context window is finite, and long conversations lose detail.

What to do

- Move decisions into a file
- One conversation per task
- New conversation: read the file first, re-run the tests

Example: at turn 40 it uses an inner join; you chose a left join at turn 3.

```
Read expectations.md. State the join type, reference
arm, and sample rules in force before you continue.
```

Correct, Restart, or Do It Yourself?

Correct it

The problem is local: one wrong line, one missed instruction, and the conversation is otherwise clean.

Say exactly what is wrong and what you expected.

Restart

Two corrections have not worked, the conversation is full of failed attempts, or earlier decisions are lost.

Commit first. Carry over only what is written in files.

Do it yourself

The task is a judgment call that is yours (specification, interpretation), or you are spending longer prompting than writing.

What makes every option cheap

A recent commit. Reverting, restarting, and taking over by hand all start from a state you can return to.

Next Session

Session 4: Power Use — Search, Swarms, Connected Tools, Skills

- Web search and literature scans, and why you check every citation
- Subagents: fan out work, use one as a verifier, and run a referee on a paper
- Connecting tools (MCP) and building a skill

Before next session:

- Install Node.js (LTS) from `nodejs.org`. Check `node --version` and `npm --version`. Session 4 installs the `claude-ignore` hook with `npm`.
- Make git a habit on one real project: commit before every Claude Code instruction
- Think of a topic in your field you could run a literature search on

Back Matter

Block	Min
A. LaTeX basics for newcomers	5
B. GitHub: vocabulary, branches, PRs, merge conflicts	15–20
C. Git extras and the standalone git practice exercise	10–20
D. *Reference only:* GitHub collaboration exercise ("Your Chapter")	—
E. If the start command fails	2
F. A reference verification protocol (compare with your own rules)	3

LaTeX Basics (For Those New to It)

```
\documentclass{article}
\usepackage{booktabs}
\title{My Research Paper}
\author{Your Name}
\begin{document}
\maketitle
\section{Data Description}
We use a survey of 2,044 Polish adults...
\section{Results}
\input{output/robustness_table.tex}
\end{document}
```

Key idea: `\input{output/robustness_table.tex}` pulls in the table from Session 2. When the table updates, the paper updates.

◂ Back matter

What Are Git and GitHub?

Git is a tool that tracks every change to every file in a project.
It runs on your computer. It is free and open source.

Think of it as "track changes" for your entire project directory — not just one document, but every file: code, data descriptions, LaTeX files, everything.

GitHub is a website that hosts git projects online.

- It is where open-source code, replication packages, and increasingly papers live
- Git works entirely on your machine. GitHub is optional — it adds backup, sharing, and collaboration
- This course uses git locally only. This block is for when you start sharing.



The Key Idea

Git lets you save snapshots of your project at any point. You can always see what changed between snapshots, and go back to any previous one.

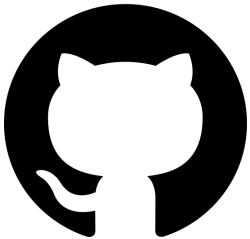
First: What Is GitHub?

GitHub is where code lives on the internet.

Every project is a *repository* — a folder with all its files plus the full history of every change.

Two copies of everything:

- **Remote** — the shared version on `github.com`
- **Local** — the version on your machine



The short version

GitHub hosts the *shared* copy. You work on your *local* copy. Git keeps them in sync.

The Vocabulary

Five words that will keep coming back today:

- `clone` — make your first local copy of a remote repo
- `pull` — fetch the latest changes *from* the remote into your local copy
- `push` — send your local changes *to* the remote
- **branch** — a parallel line of work inside the repo; changes on a branch don't affect anyone else until it's merged
- **pull request (PR)** — a proposal to merge one branch into another, with a page on GitHub for review and discussion

The flow

`clone` once. `pull` before you start. Make a **branch**, commit, `push`, open a **PR**. Someone reviews and merges.

The Course Repo

Our materials live in one GitHub repo. Everything I push goes there; everything you pull comes from there.

Let's look at the page together. Things to notice:

- The **file tree** — `session_1/`, `session_2/`, ..., `session_4/`
- The **README** — what the repo is and how to use it
- The **commit history** — every change I've made, with a message
- The **branches** — `main` is the shared one; others are work-in-progress

Why this matters for what's next

Pull conflicts happen when two people edit files in the *same* repo. To see why, we need the picture of where things live.

The Problem We Had

In earlier runs of this course, someone hit a merge conflict in almost every session.

What we did about it:

- Deleted the local repo and recloned
- Asked Claude Code to resolve the conflict markers
- Copied work to the Desktop, wiped the directory, copied back

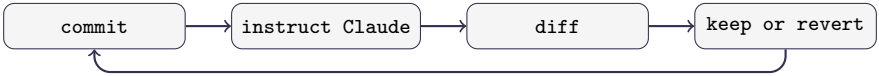
That worked. Everyone got unstuck. But it was a symptom fix.

The Root Cause

You were editing files *inside* the shared course repository. Every time I pushed new material, git had to reconcile your changes with mine. Conflicts were guaranteed by the structure, not by accident.

How GitHub Works as a Collaborator

Today's git loop is the solo loop:



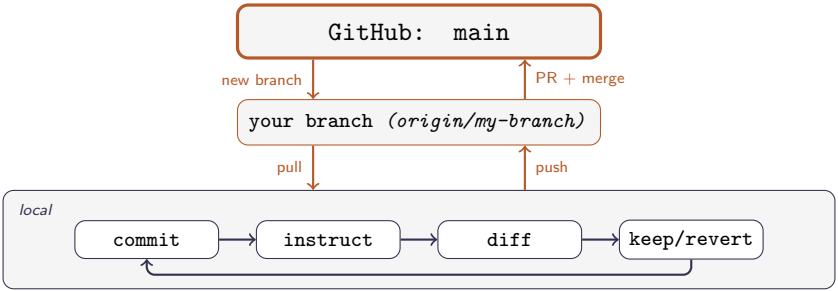
One person, one copy. Everything happens on your machine.

Add a coauthor and two new problems appear:

1. **Parallel work.** You both want to edit at once. Whose version wins?
2. **Combining work.** Your changes live on your laptop, theirs on theirs. How do they become one shared version?

GitHub's answer uses two moves: branches and pull requests.

GitHub's Answer: Branches + PRs



- `main` is the shared source of truth. Changes only return via PR + merge.
- **Your branch** is a parallel lane on GitHub — your work stays separate until reviewed.
- **Locally** is where the solo loop runs, on the files of your checked-out branch.

Working With Branches: The Commands

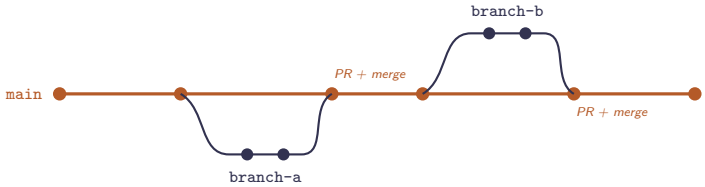
The whole branch workflow in five moves:

```
# 1. Start from an up-to-date main
git checkout main
git pull
# 2. Create a new branch
git branch my-branch-name
# 3. Switch to it
git checkout my-branch-name
# 4. Commit
git add <files>
git commit -m "..."
# 5. Push (first time uses -u; later pushes: git push)
git push -u origin my-branch-name
```

`git branch` lists branches (* marks current). Commit before switching — uncommitted edits follow you between branches.

What Is a Pull Request?

A pull request (PR) is the proposal to merge one branch into another. Everyone branches off `main`, works independently, then opens a PR to rejoin.



On the PR page: a diff, a conversation thread, and an approve-and-merge button gated by the drafter's assigned **reviewer**.

On a collaborative repo, every PR has an assigned reviewer. They have to click approve *and* merge — nothing reaches `main` until they do.

Push vs. Pull Request: Two Different Moves

These get conflated constantly, but they do different jobs.

`git push`

- Uploads your local commits to *your branch* on GitHub
- Pure sync — mechanical, instant, no review
- Touches only your branch; `main` is untouched
- You do it *many* times as you work

Pull request (PR)

- Proposes merging your branch *into* `main`
- A request — reviewed, discussed, then merged
- The *merge* is what actually changes `main`
- One per chunk of work; push again and it updates

The one-liner

Push sends your commits to your branch. **A PR** asks to fold that branch into `main`. Push moves code *to the server*; only a *merged* PR moves it *into the shared version*.

You can push a hundred times and `main` never changes — it changes only when a PR is merged.

Troubleshooting: When GitHub Fights Back

Common failure modes during the exercise — and the fix.

- **Authentication fails on first push.** Run `gh auth login` and pick HTTPS + browser. (`brew install gh` first if needed.)
- **"Updates were rejected"** — `main` moved while you worked.
  `git pull --rebase origin main && git push`
- **Conflict markers (<<<<<<<) in a file.** Open it, edit to the version you want, delete the markers. Then `git add <file>` and `git rebase --continue` (or `git commit` if merging). Walkthrough →
- **You committed to `main` by accident.** Save the work to a new branch, then rewind `main`:

```
git checkout -b chapter-N
git checkout main && git reset --hard origin/main
git checkout chapter-N
```

- **PR opened against the wrong base.** Edit the base branch in the GitHub UI at the top of the PR — no need to close and reopen.
- **Stuck for more than 90 seconds.** Flag me. Don't silently spin — the fix is usually one command.

Anatomy of a Merge Conflict

When git can't auto-combine two versions of the same lines, it leaves the file marked up like this:

```
The keynote speaker took the stage.
<<<<<<< HEAD
He launched into a 90-minute monologue about LLMs.
=======
She fumbled with the slide remote for two full minutes.
>>>>>>> origin/main
The audience applauded politely.
```

Three parts:

- `<<<<<<< HEAD` — start of **your** version (the branch you're on)
- `=======` — divider
- `>>>>>>> origin/main` — end, plus the name of the **incoming** version

Git isn't broken. It just can't tell which version you want — and is asking you to decide.

Resolving a Conflict by Hand

1. **Find them:** `git status` *(conflicted files appear under "both modified")*
2. **Open each one. Edit until only the version you want remains — no markers.**

Before:

```
<<<<<<< HEAD
He launched into a 90-min
monologue about LLMs.
=======
She fumbled with the slide
remote for two full minutes.
>>>>>>> origin/main
```

After (keep one, the other, or merge):

```
She fumbled with the slide
remote for two full minutes,
then launched into a 90-min
monologue about LLMs.
```

3. **Stage the file and continue.**

```
git add chapters/chapter_3.md && git rebase --continue
# (or: git commit, if you were merging instead of rebasing)
```

Want to bail out? `git rebase --abort` (or `git merge --abort`) reverts to the pre-conflict state.

Letting Claude Resolve It

Claude Code is good at the mechanical part — finding the markers and proposing a sensible merge.

A prompt that works:

```
There's a merge conflict in chapters/chapter_3.md.
Read both sides, propose a resolution that keeps the
intent of both, and apply it. Show me the diff first
- do not commit.
```

Safe to delegate:

- Trivial conflicts (formatting, adjacent edits)
- One side is clearly the newer or correct version
- Prose where the diff is short and you can read it end-to-end

Do it yourself:

- Code where behavior changes — you need to understand both sides
- Numerical results, tables, anything you'd want to verify line-by-line
- Conflicts touching more than a handful of files

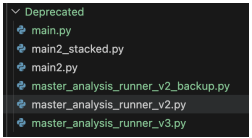
Always read the diff. "Resolved" is not the same as "correct."

Back matter

Why Researchers Should Care About Git

Without git, you have probably done this:

- `analysis_v2.py`, `analysis_v2_final.py`, `analysis_v2_final_REAL.py`
- "Which version of the cleaning script produced these results?"
- "I changed something last week and now the code is broken"

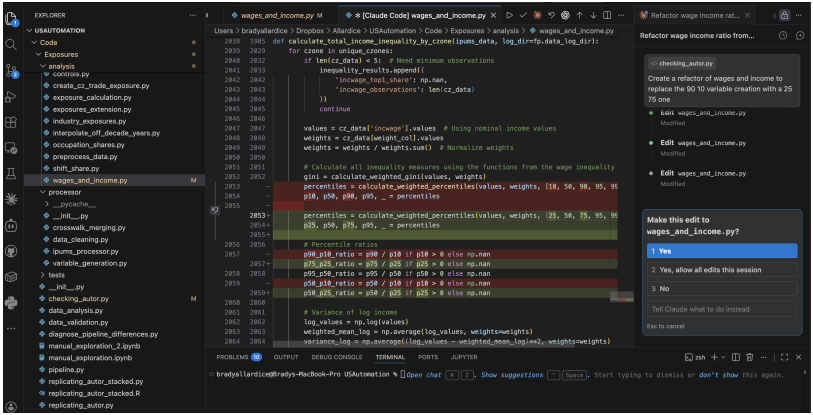


With git:

- One file, full history. Every version is recoverable.
- You can see exactly what changed, when, and why.
- You can undo any change — even from weeks ago.

Git is useful for any research project. But when you are working with an agent that modifies your files directly, it goes from useful to essential.

You Have Already Seen This



Red = removed, green = added. You have been reading diffs since Session 1.

In VS Code: When working directly in the editor, you see the `git diff` visually before you accept the change. This shows exactly what Claude Code changed.

A Typical .gitignore

.gitignore

```
# Folders
data/raw/
data/*/administrative/
output/

# Specific files
.env
.DS_Store

# Large Data Files
data_merged.csv
copy_of_internet.csv

# Python cache (auto-generated, don't need)
__pycache__/
```

Wildcards: * matches any characters.

- *.pyc = all .pyc files (anywhere in project)
- data.*.csv = data.raw.csv, data.clean.csv, etc.
- analysis_*.py = analysis_v1.py, analysis_v2.py, etc.

Folders must end with / (e.g., output/, __pycache__/)

Path scope: * = any level | ** = any depth | data_merged.csv matches anywhere | data/data_merged.csv matches only in data/.

So Where Does the Data Go?

Why git is bad for data:

- Git stores *every version* of every file. A 200MB CSV edited 10 times = 2GB of history.
- Git compares files line by line. Change one cell in a CSV and git sees the entire row as new.
- Large files slow down every git operation — commits, diffs, cloning.

For most research projects (data under a few GB):

- Keep data in a synced folder: **Dropbox**, **OneDrive**, **Google Drive**
- Your code lives in git. Your data lives next to it. Your `.gitignore` keeps them separate.
- Simple, familiar, and good enough for most social science workflows

For larger or more complex needs:

- **Object storage:** AWS S3, Google Cloud Storage — cheap, scalable
- **Data versioning:** DVC (Data Version Control) — git-like tracking for large files

You do not need these today. But if your data outgrows Dropbox, they exist.

Exercise Part 1: Choose Your Project

Option A: Your own research project

- Open your project folder in VS Code: File → New Window → File → Open Folder
- Use whatever code and data you have in that project

Option B: Practice sandbox

- On your Desktop, create a new folder: `mkdir ~/Desktop/session_2_git_practice`
- Copy Session 2 materials into it: `cp -r ~/my_workspace/session_2/* ~/Desktop/session_2_git_practice/`
- Open it in VS Code: File → New Window → File → Open Folder → select `session_2_git_practice` on Desktop

Pick one and open it in VS Code.

Exercise Part 1: Initialize and Commit (10 min)

Now, in your chosen project directory:

1. **Create a** `.gitignore` FIRST: In VS Code (File → New File → name it .gitignore) or terminal (`echo ".env" > .gitignore`). Ask Claude Code to help. Include `.env`, large data files, regenerated output, and system files.
   Important (Option B): You will find a `.env` file with my real credentials. Make sure it is in `.gitignore` so it is never committed.
2. **Initialize git:** `git init` then `git add -A && git commit -m "Initial commit"`
3. **Run:** `git log --oneline`

Exercise Part 2: The Cycle (25 min)

Do this **three times** (you decide what changes):

1. **Commit** your current state
2. **Instruct Claude Code** to make a change. Pick anything:
   - Add a control variable, refactor into functions, change sample restriction
   - Fix a bug, improve documentation, rename variables
   - Add a new analysis, create a visualization, write a summary
3. **Run** `git diff` and read what changed
4. **Decide:** keep (commit) or revert (`git checkout -- .`)

Goal: Try to revert at least once. See that you can always undo.

If you revert and regret it, you can recover the changes from your last commit using `git reflog` and `git checkout`.

Commit → Instruct → Diff → Decide

Discussion:

- Did Claude Code change any files you did not expect?
- Was the diff easy to read? What was confusing?
- Did anyone revert a change? What went wrong?

The Habit

Commit → Instruct → Diff → Decide.

This is not optional when working with an agent that modifies your files. It is the minimum responsible workflow.

Back matter

Collaboration, Live

Time to see the whole loop on a real repo. I'll walk through branches, commits, push, PR, review, and merge — on `github.com/bradyallardice/github_collaboration_practice`.

What I'll do: write a list of characters for a story. You pick which story:

1. **The Conference From Hell** — every session, break, and hallway moment at an academic conference goes sideways
2. **The Dissertation That Wrote Itself** — a PhD student's dissertation has gained sentience; each chapter is one night of it taking new liberties
3. **The Department Mouse** — a very literate mouse is stealing specific things from the department; it appears to have a plan

Watch the steps. Ask questions. I'll end with the cast pushed to `main`.

Reference only. This needs GitHub accounts and collaborator access set up in advance.

Before We Go Further: GitHub Access

Two quick things:

1. **If you haven't already emailed me your GitHub username, write it on the paper going around now.**
2. **Everyone: check your email and accept the collaboration request for the class repo.** Without it you can't push a branch or open a PR.

The invite appears as an email from GitHub, and also as a banner on the repo page if you're logged in. Click **Accept invitation**.

One minute. Shout if it doesn't show up — I'll re-send.

. . . Now You're Going to Write the Story

That repo I just pushed to? You're going to write the rest of it.

Each of you claims one of the 16 chapter slots, featuring the cast I just pushed. Write, branch, PR, merge your own chapter — all the way through. At the end, we assemble every chapter and read the story out loud.

Your chapter should feature at least one of the named characters. Beyond that, go wherever it takes you.

The writing is short on purpose. The point is the Git workflow, not the prose.

Your Chapter

I've assigned each of you a specific chapter. Open `CHAPTERS.md` at the root of the class repo — your name is next to a numbered slot. That's your chapter.

Then you run the whole loop yourself:

- Branch from `main`, write your chapter at `chapters/chapter_N.md`
- Commit, push, open a PR
- Approve and merge your own PR

Ground rules

- **Length doesn't matter.** A paragraph is fine.
- **Don't try to match other chapters.** You can't — everyone is writing simultaneously.
- **Don't polish.** Five minutes. Write, push, merge, move on.

Reference Card

1. Clone the repo, open it, branch, write, commit, push:

```
git clone https://github.com/bradyallardice/github_collaboration_practice.git
cd github_collaboration_practice
code . # open the repo in VS Code
git checkout -b chapter-N
# write your assigned chapter at chapters/chapter_N.md
git add chapters/chapter_N.md
git commit -m "Chapter N"
git push -u origin chapter-N
```

2. Open and merge the PR on GitHub:

1. Click the **Compare & pull request** banner, base = `main`
2. **Create pull request**
3. **Merge pull request** → **Confirm merge**

Something break? → Troubleshooting GitHub

What to Expect

Some of you will finish. Some will not. Both are fine.

The goal is that *everyone sees the loop end-to-end*: branch, push, PR, merge. Even if you need help at every step, you will recognize the shape next time.

What will go wrong (for some of you):

- **Push rejected** because `main` moved. Run `git pull --rebase origin main` and push again.
- **PR opened against the wrong base.** Edit the base branch in the GitHub UI, or close and reopen.
- **You get stuck for more than 90 seconds.** Flag me. Don't silently spin.

After all PRs are merged, I'll run the assembly script and read us the story.

Back matter

If the Start Command Fails

Run this in the course repo, then run the start command again (or copy the folder by hand, first time only):

```
cd ~/claude-code-workshop
git fetch origin
git reset --hard origin/student # match the published version
```

This discards edits made *inside the course repo*; your `my_workspace` is not touched. Do not use `git pull`.

Back to the ritual

Back matter

Reference Protocol (1/2)

One reference protocol, part 1. Compare it with your own rules:

```
## Verification protocol (required)
### Before implementing a step
- State the expected inputs, keys, and row counts.
- Write the verification tests first and show them to me.
- Ask me before settling any research-design choice
  (join type, reference category, sample, weights).
  Mark any choice you proposed yourself.

### Tests must be independent of the code they check
- Take expected values from the raw files, a different
  method, or values I give you. Never paste the
  pipeline's own output in as the expected value.
- Every step gets a row-count reconciliation (rows in -
  rows dropped, by reason = rows out), data-integrity
  checks, and a trace of at least 3 records from the
  raw file to the final output.
```

Reference Protocol (2/2)

Part 2 of the reference protocol:

```
### Never make a failing test pass by
- Editing expected values, deleting or loosening
  assertions, suppressing warnings, or changing the
  specification. Report the failure instead.

### Before saying a step is done
- Run the tests and show me the output.
- Report what passed, failed, and was skipped, and what
  remains unverified or needs my judgment.
- Passing tests do not show the research design is
  valid. Say what the tests do not cover.
- If you cannot write an independent test, say so.
```

Full text: `docs/verification_protocol.md` in this folder. CLAUDE.md is an instruction, not enforcement: read the tests too.

Back matter